

Lecture 07: Functions in Kotlin OOP

Parameters, `this`, named arguments, return types

Agenda

1. Functions as members (and top-level)
2. Parameters and default values
3. `this` and named arguments
4. Return types and Unit

Functions in an OOP course

You already know `fun main()`. In OOP we emphasise:

- **Member functions** — belong to a class; operate on an instance
- **Top-level functions** — file-scoped helpers (Kotlin allows these freely)
- Clear inputs (parameters) and outputs (return type)

Member vs top-level

```
1 class Account {  
2     var balance = 0  
3     fun deposit(amount: Int) { balance += amount }  
4 }  
5  
6 fun formatMoney(cents: Int): String = "${cents / 100}.${cents % 100}"  
7  
8 fun main() {  
9     val a = Account()  
10    a.deposit(500)  
11    println(formatMoney(a.balance))  
12 }
```

Single-expression functions

When the body is one expression:

- `fun double(x: Int): Int = x * 2`
- Return type can often be inferred, but **for public API prefer explicit returns**

Block body uses `return`; single-expression form uses `=`.

Quiz 1 — Which compiles?

Quiz

Which declaration is valid Kotlin?

1. `fun f(x) = x + 1`
2. `fun f(x: Int) = x + 1`
3. `fun f(Int x) = x + 1`
4. `int f(x: Int) { return x + 1 }`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 1

Answer

2. `fun f(x: Int) = x + 1`

Parameter types are required. Option 3 is Java-ish order; option 4 is not Kotlin.

Positional parameters

```
1 class Rect(val w: Int, val h: Int) {  
2     fun area(): Int = w * h  
3     fun scale(fx: Int, fy: Int): Rect = Rect(w * fx, h * fy)  
4 }  
5  
6 fun main() {  
7     val r = Rect(3, 4)  
8     println(r.scale(2, 1).area()) // 24  
9 }
```

Arguments are matched left-to-right unless you use names (next section).

Default parameter values

```
1 fun greet(name: String, polite: Boolean = true) {  
2     if (polite) println("Hello, $name")  
3     else println("Hey $name")  
4 }  
5  
6 fun main() {  
7     greet("Ada")  
8     greet("Ada", false)  
9 }
```

Defaults reduce overloads for simple variants.

Parameter design tips

- Prefer small parameter lists; group related data into a class/data class later
- Use clear names: `amount` not `a`
- Avoid boolean flag jungles — consider `enum` (Lecture 10)

Quiz 2 — Defaults

Quiz

Given `fun f(a: Int = 1, b: Int = 2) = a + b`, what is `f(10)`?

1. Compile error
2. 3
3. 12
4. 11

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. 12

Positional 10 binds to a; b keeps default 2.

The receiver: `this`

```
1 class Player(var name: String) {  
2     fun rename(name: String) {  
3         this.name = name    // property vs parameter  
4     }  
5 }
```

- `this` — current instance (receiver)
- Required when a parameter shadows a property
- Often omitted when names do not clash

Named arguments

```
1 fun createUser(  
2     name: String,  
3     email: String,  
4     admin: Boolean = false  
5 ) { /* ... */ }  
6  
7 fun main() {  
8     createUser(  
9         email = "a@ex.com",  
10        name = "Ada",  
11        admin = true  
12    )  
13 }
```

Names document call sites and allow skipping earlier defaults carefully.

Named + default interplay

- After the first named argument, remaining args should be named (mix rules: once you name one, name the following ones)
- Call `f(b = 3)` to set only `b` when `a` has a default

Common mistake

Assuming Java-style overload explosion is required. In Kotlin, defaults + named args often replace several overloads.

Quiz 3 — this

Quiz

Inside `fun set(x: Int) { this.x = x }` on a class with property `x`, what does `this.x` mean?

1. The parameter `x`
2. A local variable in `main`
3. The property `x` of the current instance
4. Always `null`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. The property `x` of the current instance.

`this` disambiguates the property from the parameter of the same name.

Return types

- Declare with `:` Type after the parameter list
- `Unit` — “no meaningful value” (like `void`); may be omitted
- Use `return expr` in block bodies
- For expressions: `fun f(): Int = 42`

Returning values from methods

```
1 class BankAccount(var balance: Int) {  
2     fun withdraw(amount: Int): Boolean {  
3         if (amount > balance) return false  
4         balance -= amount  
5         return true  
6     }  
7 }  
8  
9 fun main() {  
10     val a = BankAccount(100)  
11     println(a.withdraw(40)) // true  
12     println(a.withdraw(100)) // false  
13 }
```

Expression-body vs block

```
1 fun abs(x: Int): Int = if (x >= 0) x else -x
2
3 fun absBlock(x: Int): Int {
4     return if (x >= 0) x else -x
5 }
```

Prefer expression style for short pure functions; use blocks when there are multiple steps or early returns.

Quiz 4 — What prints?

Quiz

What does this print?

```
1 fun f(x: Int): Int {  
2     if (x < 0) return -x  
3     x + 1    // suspicious  
4     return x  
5 }  
6 fun main() = println(f(3))
```

1. 4
2. 3
3. Compile error
4. -3

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. 3

$x + 1$ is evaluated and discarded; the function returns x . (Not an expression-body function.)

Mid-level trap: unused expressions.

Summary

- Member functions act on instance state; top-level helpers remain useful
- Parameters need types; defaults and named args clarify calls
- `this` is the receiver — essential under shadowing
- Declare return types; `Unit` for no value; don't discard intended results

Looking ahead

Next: constructors — secondary constructors, overloading, and the primary constructor.